

SP

STUDENTPULSE

Student Events Portal

Technical Documentation

Internet Programming II · Group Project

Core technologies: HTML · PHP · MySQL

Institution	
Group member 1	
Group member 2	
Group member 3	
Group member 4	
Group member 5	

Submission date: _____

1. Executive Summary

Purpose, users and main value

StudentPulse is a secure, database-driven web application that centralizes campus event discovery and participation. Students create accounts, authenticate, browse published events, monitor live countdowns, RSVP, view their profile and submit messages to the events team. Administrators use protected management pages to create, edit and delete event records.

The project meets the assignment requirement for at least five interlinked authenticated pages while extending the minimum scope with role-based administration, event RSVPs, CSRF protection and dynamic countdowns. The interface uses a responsive modern-blue visual system for clear navigation on desktop and mobile.

Objectives

- Provide secure registration, login, session maintenance and logout.
- Store and retrieve normalized user, event, RSVP and contact-message data.
- Validate all form input server-side and safely encode all displayed data.
- Give students a clear events dashboard and administrators controlled event management.
- Supply reproducible installation instructions, sample data and a defense guide.

Technology stack

Layer	Technology	Responsibility
Presentation	HTML5, CSS3, JavaScript	Responsive interface, forms and countdowns
Application	PHP 8+	Validation, sessions, authorization and request handling
Data	MySQL / MariaDB	Persistent relational records and constraints
Data access	PDO	Prepared statements and controlled database errors

2. Requirements Traceability

How the solution addresses the project specification

Requirement	StudentPulse implementation
User accounts	Unique username/email, hashed password and student/admin role in users table.
Registration	Server validation, duplicate check, password hashing and prepared INSERT.
Authentication	Email/password verification, session regeneration and role storage.
Five protected pages	Dashboard, Profile, Contact, Message Submissions and Events Hub.
Contact form	Name, email, subject and message stored in contact_messages.
Submissions directory	Escaped records displayed in a responsive HTML table.
Information hub	Events Hub retrieves event data and supports RSVP interaction.
Database export	database.sql creates schema, constraints, indexes and sample records.
Creativity	Live countdowns, RSVP workflow, role-based event CRUD and responsive UI.

User roles

Student: registers, logs in, views the dashboard and profile, browses events, adds or cancels an RSVP, submits messages and views the submissions directory.

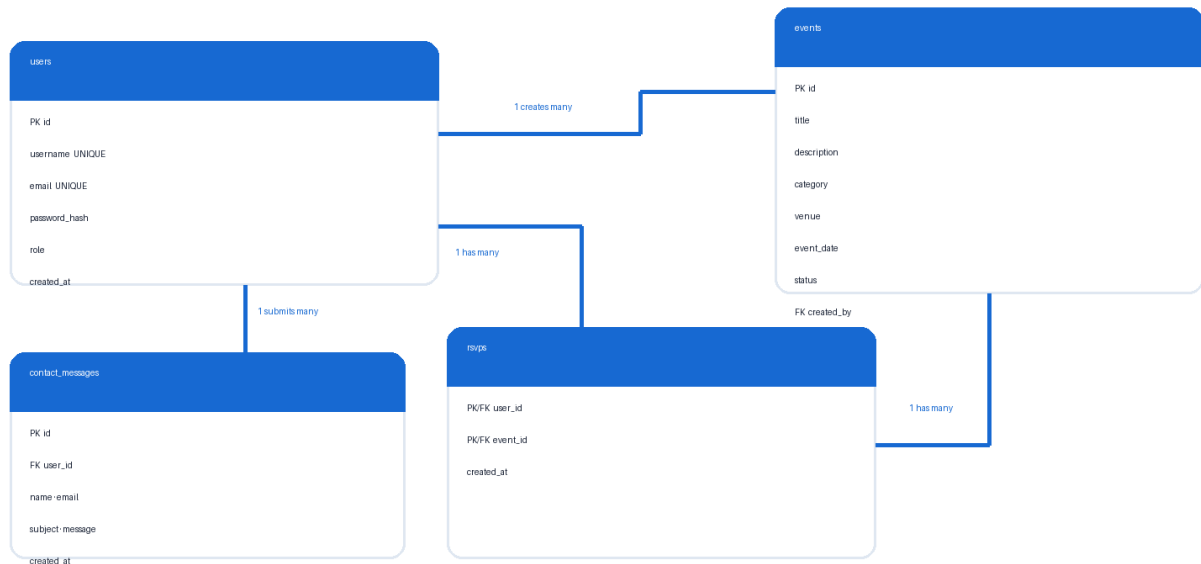
Administrator: has every student capability plus access to event creation, editing and deletion. Each administrator route calls admin() before displaying content.

3. Database Design

Normalized entities and referential integrity

StudentPulse database relationships

Normalized relational structure - PK = primary key; FK = foreign key



Normalization: account data is stored once in `users`; event data is stored once in `events`; contact messages are independent transactions; and the many-to-many relationship between users and events is resolved by `rsmps`. The composite RSVP primary key prevents duplicate attendance records.

Key constraints

- `users.username` and `users.email` are UNIQUE.
- Foreign keys preserve valid relationships and define safe cascade/set-null behavior.
- `rsmps` uses `(user_id, event_id)` as a composite primary key.
- Indexes support event date/status filtering and chronological message retrieval.

4. System Architecture

Request, session and database flow

System architecture and request flow

A three-layer PHP application with server-side access control



Request lifecycle

1. The browser requests a PHP page and sends form values when applicable.
2. Protected pages call `auth()`; admin routes additionally call `admin()`.
3. POST requests verify the CSRF token before processing.
4. PHP trims and validates values, then executes a PDO prepared statement.
5. MySQL applies unique, foreign-key and type constraints.
6. Returned values are encoded with `e()`, which wraps `htmlspecialchars()`, before insertion into HTML.

Separation of concerns

`config/database.php` owns database connectivity. `includes/core.php` centralizes authentication, escaping, CSRF, redirects and flash messages. `includes/header.php` and `footer.php` maintain a consistent layout. Individual pages contain focused request logic and views.

5. Security, Validation & Sanitization

Controls applied to common web risks

Risk	Control in StudentPulse
SQL injection	PDO prepared statements with bound positional values; emulated prepares disabled.
Password disclosure	password_hash() stores a one-way adaptive hash; password_verify() checks login.
Cross-site scripting	e() applies htmlspecialchars(..., ENT_QUOTES, UTF-8) to dynamic output.
Session fixation	session_regenerate_id(true) runs after successful authentication.
Unauthorized access	auth() and admin() stop unauthenticated/unauthorized requests.
Cross-site request forgery	Random per-session CSRF token verified on every state-changing form.
Duplicate records	UNIQUE keys and composite RSVP primary key enforce integrity.
Information leakage	Database exceptions are logged; a generic connection message is shown.

Validation examples

- Registration: username 3–50 characters, valid email, password at least 8 characters and matching confirmation.
- Contact: name 2–100, valid email, subject 3–150 and message 10–2,000 characters.
- Event management: bounded title/description/venue, valid date, allow-listed status and integer IDs.

Sanitization is context-specific: prepared statements protect SQL context; HTML escaping protects output context. Input is also trimmed and length-limited, but trimming alone is not treated as a security control.

6. Installation Guide

Local setup with XAMPP

1. Install XAMPP and start Apache and MySQL.
2. Copy the StudentPulse folder into C:\xampp\htdocs\.
3. Open <http://localhost/phpmyadmin> and select Import.
4. Choose database.sql and run the import. It creates the studentpulse database and sample records.
5. Open config/database.php and confirm the database credentials. XAMPP normally uses user root with a blank password.
6. Visit <http://localhost/StudentPulse/>.
7. Log in as admin@studentpulse.test or student@studentpulse.test with password password.
8. Change the sample passwords before a real deployment.

Project structure

Path	Purpose
assets/	Responsive CSS and countdown JavaScript
config/database.php	Central PDO connection
includes/	Shared security helpers, navigation and footer
admin/	Role-protected event management
database.sql	Schema, constraints and sample records
documentation/	PDF report and defense guide
*.php	Authentication and application pages

Configuration note

If MySQL uses a different port, password or host, update only config/database.php. The application requests utf8mb4 and PDO exception mode to preserve text correctly and surface development errors through server logs.

7. Test Plan & Expected Results

Functional and security verification

Test	Expected result
Register valid student	Account inserted; user redirected to login with success message.
Register duplicate email	No insert; a clear duplicate warning appears.
Submit invalid email	Server rejects input and retains safe field values.
Log in correctly	Session ID regenerates; dashboard shows username.
Open dashboard while logged out	Request redirects to login.php.
Student opens /admin/events.php	Request redirects away from admin area.
Submit contact form	Message appears in submissions table.
RSVP twice	Composite key prevents duplicate rows.
Render stored <script> text	Text displays harmlessly; browser does not execute it.
Send POST without CSRF token	Server returns HTTP 403.
Log out	Session data and session cookie are destroyed; login page opens.
Mobile viewport	Navigation collapses and cards/forms remain readable.

Known deployment considerations

- Use HTTPS and secure cookie flags on an internet-facing server.
- Move database secrets into environment configuration for production.
- Add rate limiting or temporary lockout for repeated failed logins.
- Restrict the public submissions directory if messages may contain sensitive information.

8. Demonstration & Group Defense

Recommended presentation sequence

1. Register a new account and explain each validation rule.
2. Log in and show the protected dashboard and username session value.
3. Open Events Hub, explain the database query, countdown and RSVP table.
4. Submit Contact Us, then verify the row in Message Submissions and phpMyAdmin.
5. Log out; log in as admin and demonstrate create/edit/delete.
6. Open database.sql or the ERD and explain foreign keys and normalization.

Member responsibilities

Member	Registration number	Primary area
		Database and ERD
		Authentication and sessions
		Events and RSVP logic
		Interface and responsive CSS
		Testing, documentation and presentation

Submission checklist

- Replace all name/registration-number placeholders.
- Run the application locally and repeat every test in Section 7.
- Confirm StudentPulse/, database.sql and the PDF report are inside the ZIP.
- Ensure every member can explain prepared statements, password hashing, sessions, XSS protection and their assigned code.